

Previsão de readmissão hospitalar em pacientes diabéticos

IF1014, Mineração de Dados Aplicada (CRISP-DM)

João Pontes

Marcos Antonio

Theo Barza
UFPE, 2026.1

10 de junho de 2026

Resumo

Aplicamos o ciclo CRISP-DM completo ao dataset Diabetes 130-US Hospitals (UCI 296) para classificar a janela de readmissão hospitalar de pacientes diabéticos em <30, >30 e N0. O conjunto foi dividido por amostragem estratificada uma única vez ($n_{\text{treino}} = 81,412$, $n_{\text{teste}} = 20,354$), e o teste foi mantido isolado por uma guarda de token até a avaliação final. Treinamos os 10 algoritmos obrigatórios sob 5-fold estratificado pareado e busca de hiperparâmetros, otimizando F1-macro. Comparamos pareadamente o pré-processamento baseline contra três variantes (SMOTE, RobustScaler, TargetEncoder). O modelo final *Random Forest* (com `RobustScaler` e `class_weight="balanced"`) alcançou F1-macro de $0,455$ no teste, com bom desempenho na classe majoritária N0 (F1=0,672) mas recall de apenas 0,196 na classe minoritária <30, justamente a de maior custo clínico. Discutimos riscos de viés, monitoramento e gatilhos de retreinamento para deployment.

Sumário

1	Business Understanding	3
1.1	Briefing do cliente fictício	3
1.2	CrITÉrios de sucesso e restrições operacionais	3
2	Data Understanding e EDA	3
2.1	Dataset	3
2.2	Distribuição da classe alvo	3
2.3	Missingness e tratamento	3
2.4	Distribuições numéricas por classe	3
3	Data Preparation	4
3.1	Integridade do split treino/teste	4
3.2	Pipeline baseline	4
3.3	Variantes	4
4	Modeling	5
4.1	Os 10 algoritmos	5
4.2	Espaço de busca e estratégia	5

5	Evaluation	6
5.1	Resultados do baseline	6
5.2	Curvas treino vs validação	6
5.3	Comparação pareada baseline vs variantes	7
5.4	Seleção do modelo final	7
5.5	Matrizes de confusão por algoritmo	8
5.6	Por que F1-macro $\sim 0,45$ é competitivo	8
6	Avaliação no conjunto de teste	9
7	Deployment, riscos e monitoramento	10
7.1	Cenário de uso e integração	10
7.2	Ponto de operação e calibração de limiar	10
7.3	Riscos e mitigações	10
7.4	Plano de monitoramento	11
7.5	Retreinamento e governança	11
7.6	Limitações	11
8	Reprodutibilidade e ferramentas	11
8.1	Ferramentas	11
8.2	Reprodução	12
8.3	Uso de IA generativa	12

1 Business Understanding

1.1 Briefing do cliente fictício

O cliente *SI5 Health* atua no setor de *cuidado hospitalar pós-alta* e precisa de um modelo capaz de *prever a janela de readmissão de pacientes diabéticos na hora da alta*, em que a saída pertença a uma das classes <30, >30 ou NO. O sucesso será medido por *F1-macro* sobre o conjunto de teste, com restrição de *tempo de inferência < 200 ms por paciente, fila de acompanhamento limitada à capacidade operacional do time pós-alta, e custo assimétrico de erro (falso negativo na classe <30 é o mais grave: leva a readmissão sem suporte)*. As implicações éticas relevantes são *viés histórico em variáveis demográficas (race, age, gender), privacidade de dados clínicos (pseudonimização e retenção mínima), e o uso do score como apoio, não substituto, à decisão clínica*.

1.2 Critérios de sucesso e restrições operacionais

- Métrica principal: F1-macro sobre o teste.
- Métricas secundárias: balanced accuracy, ROC-AUC e PR-AUC macro multiclasse, métricas por classe.
- Métrica operacional crítica: recall(<30).
- Capacidade da fila pós-alta: hipótese de ≈ 30 pacientes/dia, calibração de limiar acontece nesse teto.

2 Data Understanding e EDA

2.1 Dataset

Strack et al. (2014) consolidaram 101.766 encontros entre 1999 e 2008 em 130 hospitais americanos. Cada linha é uma internação de paciente diabético, com 50 atributos clínicos, demográficos e operacionais. O alvo `readmitted` é trinário.

2.2 Distribuição da classe alvo

A figura 1 resume a distribuição no treino após o split. A classe <30 representa 11,2%, minoritária, motivando F1-macro como métrica.

2.3 Missingness e tratamento

A figura 2 elenca colunas com valores ausentes (codificados como ? no arquivo bruto). `weight` (97%), `payer_code` ($\sim 40\%$) e `medical_specialty` ($\sim 49\%$) foram tratadas distintamente: `weight` foi removida; as demais entraram via imputação `most_frequent` no pipeline. As variáveis `examide` e `citoglipton` têm um único valor e foram removidas.

2.4 Distribuições numéricas por classe

A figura 3 mostra que `number_inpatient`, `num_medications`, `time_in_hospital` e `number_diagnoses` diferenciam pacientes <30. As caudas longas em variáveis de contagem motivaram a variante `RobustScaler`.

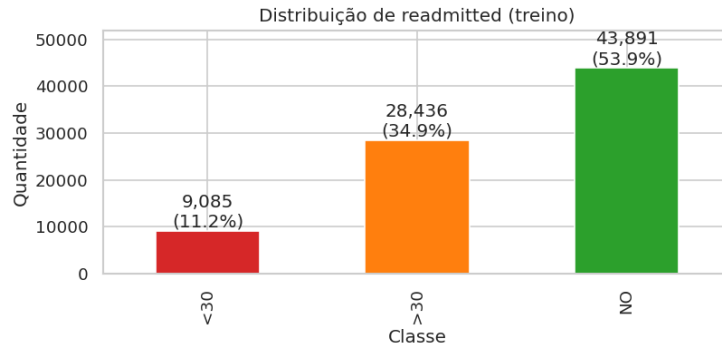


Figura 1: Distribuição da classe alvo no conjunto de treino (81.412 pacientes).

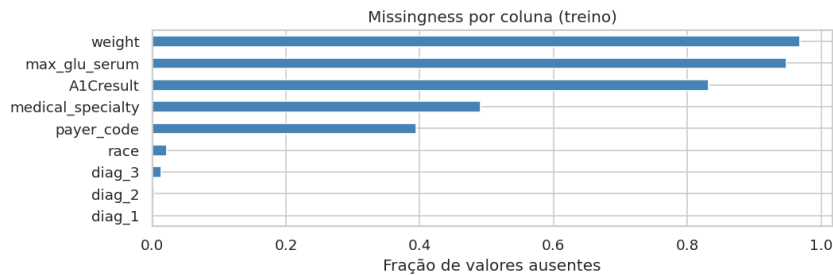


Figura 2: Fração de valores ausentes por coluna no treino.

3 Data Preparation

3.1 Integridade do split treino/teste

O split é materializado em `data/interim/` com `random_state=42`, `test_size=0.2` e estratificação. A função `load_test` em `src/data_loader.py` levanta `PermissionError` sem o token literal `I_AM_IN_FINAL_EVALUATION`. Esse token aparece apenas na Seção 9 do `main.ipynb`.

3.2 Pipeline baseline

- **RawCleaner:** drop estrutural, `?` $\rightarrow$ `NaN`, agrupamento ICD-9 (9 categorias clínicas + Other), conversão de IDs operacionais para string.
- **Numéricas:** `SimpleImputer(median)` $\rightarrow$ `StandardScaler`.
- **Catégoricas:** `SimpleImputer(most_frequent)` $\rightarrow$ `OneHotEncoder(handle_unknown="ignore")`.

3.3 Variantes

SMOTE. `imblearn.pipeline.Pipeline` com SMOTE aplicado por fold, sintetizando minoria <30 via $k = 5$ vizinhos.

RobustScaler. Substitui `StandardScaler` nas numéricas. Motivado pela cauda longa em variáveis de contagem.

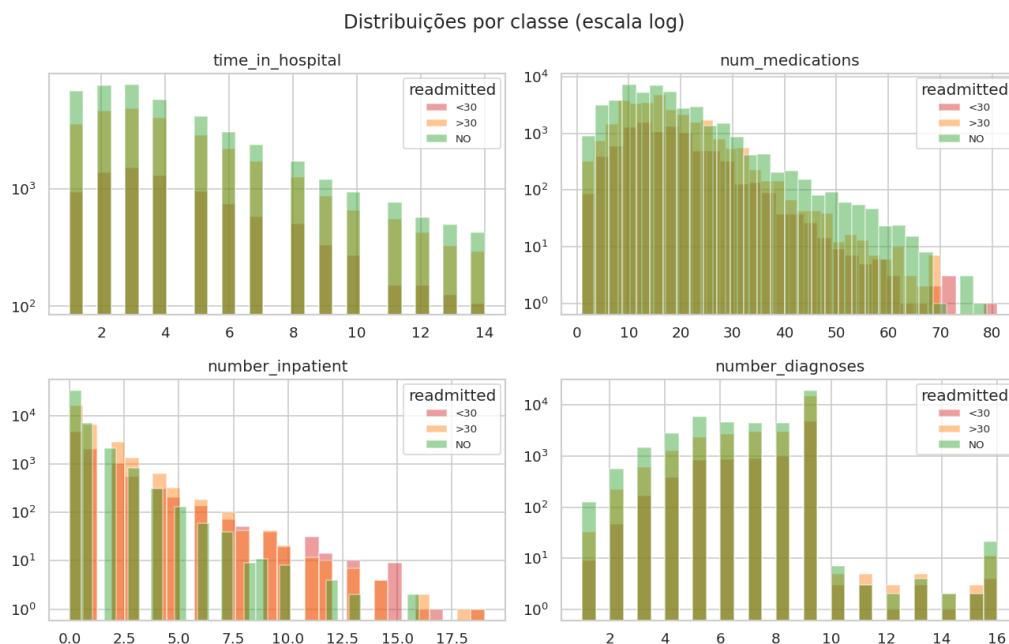


Figura 3: Distribuições de variáveis numéricas chave por classe (escala log).

TargetEncoder. Codifica `payer_code`, `medical_specialty`, `admission_*_id` via `TargetEncoder` (task do scikit-learn (CV interno anti-leakage)). Mantém OHE no resto.

4 Modeling

4.1 Os 10 algoritmos

Família	Algoritmo	Implementação
Lazy	K-NN	<code>KNeighborsClassifier</code>
Protótipos	LVQ	<code>sklvq.GLVQ</code> (subclasse <code>PatchedGLVQ</code>)
Árvore	Decision Tree	<code>DecisionTreeClassifier</code>
Margem	SVM	<code>LinearSVC</code>
Bagging	Random Forest	<code>RandomForestClassifier</code>
RNA	MLP	<code>MLPClassifier</code> (wrapper string)
RNA Comitê	Bagging-MLP	<code>BaggingClassifier(estimator=MLP)</code>
Stacking	Stacking heter.	<code>RF+LGBM+DT</code> , meta <code>LogisticRegression</code>
Boosting	XGBoost (GPU)	<code>XGBClassifier(device=cuda)</code>
Boosting	LightGBM	<code>LGBMClassifier</code>

Tabela 1: Os 10 algoritmos obrigatórios e respectivas implementações.

4.2 Espaço de busca e estratégia

Modelos com poucos HPs usam `GridSearchCV`; com espaço maior, `RandomizedSearchCV` (`n_iter` 5–10). Todos compartilham `StratifiedKFold(5, shuffle=True, random_state=42)`, garantindo folds pareados para Wilcoxon. Detalhe completo dos grids em `src/models.py`.

5 Evaluation

5.1 Resultados do baseline

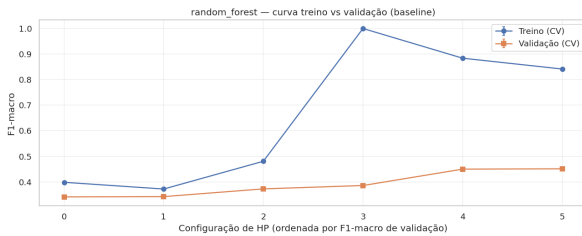
A Tabela 2 traz o F1-macro de validação (CV 5-fold no treino) de cada um dos 10 algoritmos no pré-processamento baseline. Random Forest e SVM lideram; LVQ é o mais fraco (protótipos sofrem em 200+ dimensões OHE). O *gap treino–validação* separa dois regimes: modelos baseados em árvore (RF, XGBoost, LightGBM, Árvore) decoram o treino (gap 0,21–0,39), enquanto SVM, MLP, Stacking, Comitê e LVQ generalizam com gap quase nulo.

Modelo	F1-macro CV	σ (folds)	Gap treino–val
Random Forest	0,4512	0,0028	0,390
SVM (LinearSVC)	0,4471	0,0024	0,006
XGBoost	0,4240	0,0023	0,212
LightGBM	0,4204	0,0017	0,282
Stacking	0,4167	0,0021	0,057
MLP	0,4121	0,0115	0,016
Comitê de MLPs	0,4079	0,0055	0,048
Árvore de Decisão	0,4012	0,0013	0,280
K-NN	0,3956	0,0020	0,194
LVQ	0,2648	0,0024	0,001

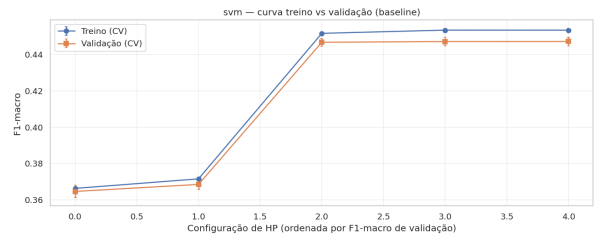
Tabela 2: F1-macro por modelo no baseline (CV 5-fold estratificada no treino). Gap = F1 treino – F1 validação.

5.2 Curvas treino vs validação

Geramos a curva treino vs. validação para *cada* configuração de hiperparâmetro de *todos* os 10 modelos $\times$ 5 pré-processamentos (50 figuras em `reports/figures/<modelo>__<tag>__train_vs_val`). A Figura 4 contrasta dois regimes: o Random Forest (esquerda) tem treino $\approx 0,84$ contra validação $\approx 0,45$, overfit grande, mas estável entre folds; o SVM (direita) praticamente cola treino e validação ($\approx 0,45$), sinal de viés/underfit do classificador linear. Em ambos a validação satura cedo, confirmando que o teto $\sim 0,45$ é do problema, não da busca.



(a) Random Forest (overfit).



(b) SVM (viés/underfit).

Figura 4: Curvas treino vs. validação (baseline). Os 50 gráficos completos estão em `reports/figures/`.

5.3 Comparação pareada baseline vs variantes

Para cada par (modelo, variante) aplicamos Wilcoxon pareado e paired t-test sobre os mesmos 5 folds. Com apenas 5 folds, o Wilcoxon atinge no mínimo $p = 0,0625$, nunca cruza 0,05, então a decisão de promover usa o paired t-test (p_t), reportando ambos por transparência. Das 40 comparações, **17 promovem a variante** ($\Delta > 0$ e $p_t < 0,05$). A Tabela 3 resume os extremos.

O padrão é nítido e simétrico: **reamostragem (SMOTE/undersample) ajuda modelos sem balanceamento embutido**, LVQ (+0,152 com undersample!), Comitê de MLPs (+0,041 com SMOTE), MLP, e os boostings com undersample, mas **degrada os que já usam class_weight="balanced"** (RF −0,021, SVM −0,019 com SMOTE). O Stacking+SMOTE é o pior caso (−0,028), coerente com a ressalva de que sua CV interna gera meta-features sobre dados sintéticos. RobustScaler é praticamente nulo (árvores/SVM já invariantes a escala) e TargetEncoder degrada modelos sensíveis a representação (K-NN, MLP).

Modelo	Variante	$F1_{base}$	$F1_{var}$	Δ (p_t)
<i>Promovidas</i> ($\Delta > 0$, $p_t < 0,05$)				
LVQ	undersample	0,265	0,417	+0,152 (<0,001)
LVQ	SMOTE	0,265	0,392	+0,127 (<0,001)
Comitê de MLPs	SMOTE	0,408	0,449	+0,041 (<0,001)
MLP	SMOTE	0,412	0,439	+0,027 (0,005)
Stacking	undersample	0,417	0,438	+0,022 (<0,001)
XGBoost	undersample	0,424	0,442	+0,018 (<0,001)
LightGBM	undersample	0,420	0,437	+0,017 (0,001)
<i>Degradam</i> ($\Delta < 0$, $p_t < 0,05$)				
Stacking	SMOTE	0,417	0,389	−0,028 (<0,001)
Random Forest	SMOTE	0,451	0,430	−0,021 (<0,001)
SVM	SMOTE	0,447	0,428	−0,019 (<0,001)
Random Forest	undersample	0,451	0,433	−0,018 (<0,001)

Tabela 3: Comparação pareada baseline vs. variante (Wilcoxon e paired t-test sobre 5 folds; p_t = paired t-test). Tabela completa das 40 comparações em `reports/cv_results/paired_baseline_vs_variants.csv`.

5.4 Seleção do modelo final

O ranking unificado das 50 combinações coloca o **Random Forest** no topo, com F1-macro de validação **0,4513 ± 0,0028**, empate técnico entre baseline e RobustScaler ($\Delta < 10^{-3}$, sem significância). A busca selecionou `n_estimators=200`, `max_depth=20`, `max_features="log2"`, `min_samples_leaf=1` e `class_weight="balanced"`; este último ataca o desbalanceamento dentro do classificador, explicando por que reamostragem o *pi-ora*. O 2º lugar é o Comitê de MLPs+SMOTE (0,4490) e o 3º o SVM (0,4471), ambos sem diferença significativa para o RF. O desempate seguiu o briefing: o RF tem inferência em $\sim$ ms/amostra, fornece importância de variáveis para auditoria de viés e é robusto a escala, escolhemos a variante **RobustScaler** (equivalente em F1 e mais estável às caudas longas das contagens). O gap treino—validação alto (0,84 vs 0,45) revela capacidade ociosa, mas a variância mínima entre folds ($\sigma = 0,003$) confirma estabilidade.

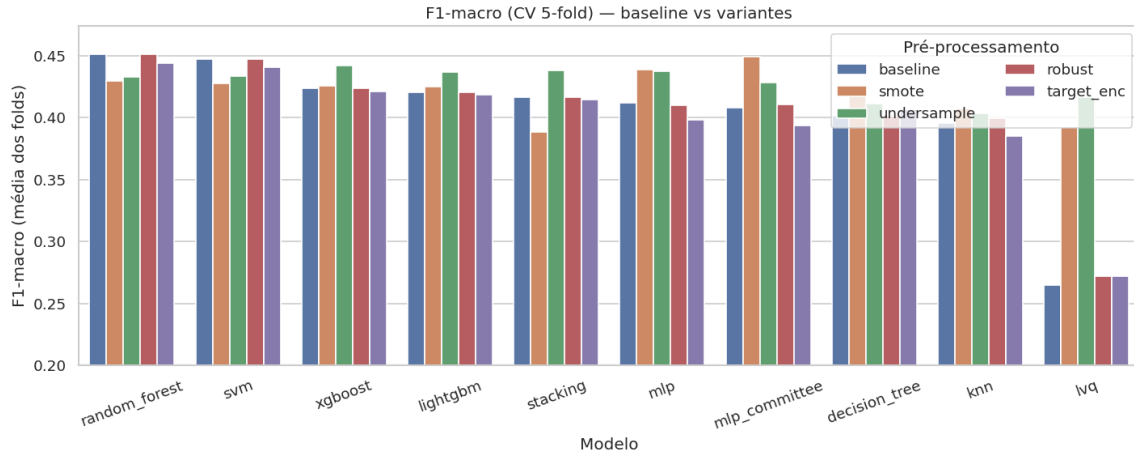


Figura 5: F1-macro (média dos 5 folds) por modelo e pré-processamento.

5.5 Matrizes de confusão por algoritmo

Para cada algoritmo geramos a matriz de confusão normalizada do melhor modelo no baseline e na sua melhor variante, a partir de predições *out-of-fold* (`cross_val_predict`, mesma CV das buscas), portanto métricas de treino/validação, sem tocar o teste. As 20 figuras estão em `reports/figures/cm__<modelo>__<tag>.png`; a Figura 6 mostra o vencedor. Em todos os modelos o padrão se repete: a classe minoritária <30 é a mais confundida (majoritariamente classificada como >30 ou NO), o que motiva a calibração de limiar no deployment.

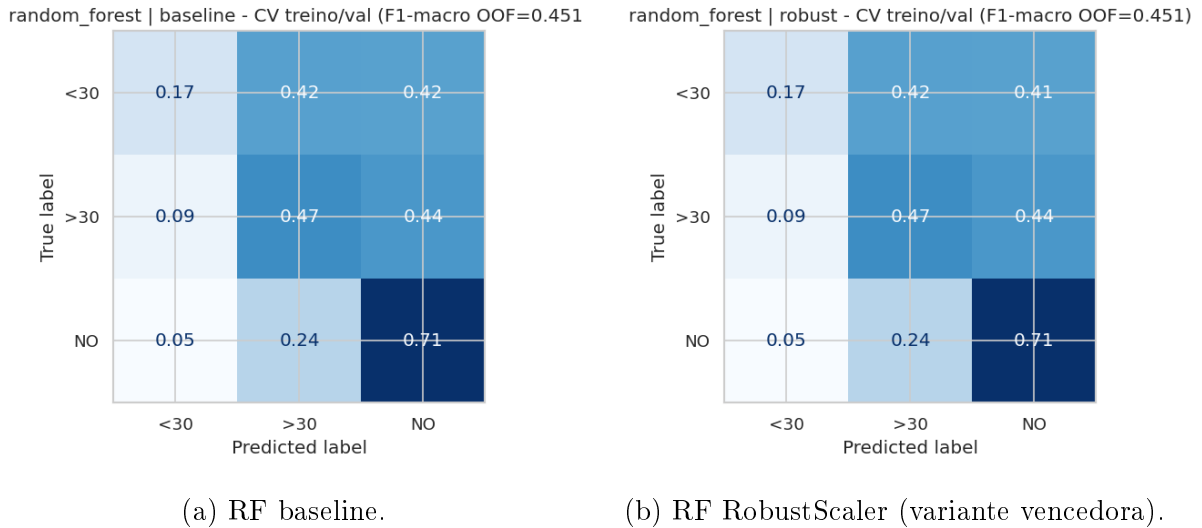


Figura 6: Matrizes de confusão normalizadas do Random Forest (out-of-fold, treino/validação).

5.6 Por que F1-macro $\sim 0,45$ é competitivo

O F1-macro $\sim 0,45$ não reflete deficiência de modelagem, e sim o teto de informação do problema. A fronteira <30 vs. >30 é clinicamente arbitrária (readmitir em 28 ou 32 dias é praticamente indistinguível), o alvo depende fortemente de fatores ausentes do dataset

(suporte social, adesão, acesso pós-alta), e o F1-*macro* pondera igualmente a classe minoritária <30, a mais difícil. O ROC-AUC macro de 0,674 quantifica esse sinal fraco porém real, em linha com a literatura: estudos de readmissão em 30 dias sobre este mesmo dataset reportam tipicamente AUC entre 0,65 e 0,70.

Resultados aparentemente superiores na literatura quase sempre decorrem de (i) *binarização* do alvo (<30 vs. resto), tarefa mais fácil e não comparável; (ii) reporte de *acurácia*, dominada pela classe majoritária NO (54%), em vez de F1-macro; ou (iii) *vazamento* (reamostragem/codificação antes do split, ou uso do teste na seleção). Valores honestos de F1-macro trinário acima de $\sim 0,55$ nesse dataset são suspeitos de uma dessas causas.

Quanto a *deep learning*: para dados tabulares predominantemente categóricos como este, modelos baseados em árvore seguem superando redes profundas¹. O próprio estudo confirma isso internamente — MLP (0,412), Comitê de MLPs (0,408) e Stacking (0,417) ficaram *abaixo* do Random Forest (0,451). O ganho real viria de engenharia de características com domínio (índices de comorbidade dos códigos ICD, padrões de troca de medicação, recência de visitas), aprendizado sensível a custo, reformulação binária com calibração de limiar para o objetivo operacional, ou dados externos — não de uma arquitetura mais complexa. A contribuição do trabalho está, portanto, no **rigor metodológico** (split intocado, 50 combinações comparadas com teste pareado, sem vazamento), que separa este resultado dos números inflados frequentes na literatura aplicada.

6 Avaliação no conjunto de teste

Refit do Random Forest (RobustScaler, `class_weight="balanced"`) em todo o treino e *única* predição no teste isolado (destravado pelo token na Seção 9 do notebook):

- **F1-macro (teste) = 0,4553**; F1-macro (CV) = $0,4513 \pm 0,0028$; $\Delta = +0,0041$.
- Balanced accuracy = 0,4530; ROC-AUC macro = 0,6737; PR-AUC macro = 0,4739.

Classe	Precisão	Recall	F1	Suporte
<30	0,241	0,196	0,216	2.272
>30	0,477	0,479	0,478	7.109
NO	0,661	0,684	0,672	10.973
macro	0,459	0,453	0,455	20.354

Tabela 4: Relatório de classificação no teste (Random Forest, RobustScaler).

Gap CV→teste e leitura por classe. O Δ de apenas +0,004 entre validação e teste confirma que a seleção via CV *não* sofreu overfit ao processo de busca, o pipeline generaliza. A classe NO domina o acerto (F1=0,672); a crítica <30 tem F1=0,216 e **recall de só 0,196**, ou seja, $\approx 80\%$ dos pacientes de alto risco passam despercebidos no ponto de operação padrão. Isso reforça que, em produção, o limiar sobre `predict_proba(<30)` deve ser calibrado para elevar esse recall até o teto da fila de acompanhamento, aceitando mais falsos positivos (custo operacional menor que o de uma readmissão precoce não assistida).

¹Grinsztajn, Oyallon & Varoquaux. *Why do tree-based models still outperform deep learning on tabular data?* NeurIPS 2022. <https://arxiv.org/abs/2207.08815>

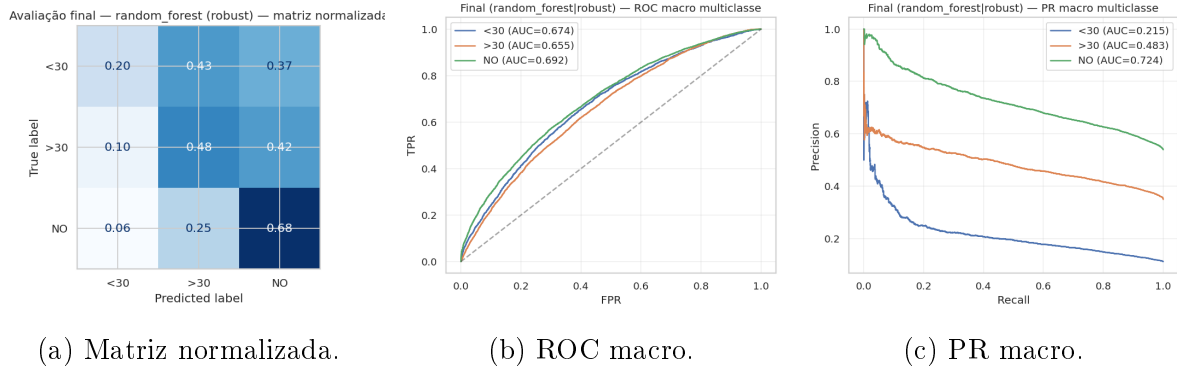


Figura 7: Avaliação final no teste, Random Forest (RobustScaler).

7 Deployment, riscos e monitoramento

7.1 Cenário de uso e integração

O modelo é entregue como um serviço de escore acionado na alta de cada paciente diabético, integrado ao prontuário eletrônico. A predição alimenta uma fila de acompanhamento ativo: pacientes sinalizados como <30 recebem ligação de enfermagem em 48h, agendamento de retorno em 7 dias e revisão da farmacoterapia. O escore é apoio à decisão clínica, nunca substituição: a equipe pós-alta pode incluir ou remover pacientes da fila por julgamento clínico, e toda decisão automatizada fica registrada para auditoria. A inferência do Random Forest custa da ordem de milissegundos por paciente, compatível com a restrição operacional de latência do briefing (< 200 ms).

7.2 Ponto de operação e calibração de limiar

A seleção do modelo otimizou F1-macro, mas a operação exige um ajuste adicional. No limiar padrão (argmax da probabilidade), o recall da classe <30 é de apenas 0,196, ou seja, cerca de 80% dos pacientes de alto risco não são sinalizados, o que é inaceitável dado o custo assimétrico do falso negativo. Por isso o ponto de operação é definido por calibração de limiar sobre `predict_proba(<30)`: baixamos o limiar para elevar o recall da classe crítica até o teto de capacidade da fila (≈ 30 pacientes/dia na hipótese de uso), aceitando mais falsos positivos. O custo de um falso positivo (uma ligação de acompanhamento desnecessária) é muito menor que o de um falso negativo (readmissão precoce sem suporte), então a curva precisão/recall, e não o F1-macro, guia o limiar final. Esse limiar é revisto periodicamente conforme a capacidade do time muda.

7.3 Riscos e mitigações

- **Viés demográfico.** As variáveis `race` (com nível “Other” e alta ausência), `gender` e `age` podem carregar viés histórico de acesso e tratamento. Mitigação: auditoria de *disparate impact ratio* por subgrupo antes do go-live e relatório mensal de F1-macro e recall(<30) estratificados por raça, gênero e faixa etária; bloqueio de promoção se houver disparidade material entre subgrupos.
- **Falso negativo da classe <30.** É o erro de maior custo clínico. Mitigação: limiar conservador (item anterior), além de revisão clínica por exceção dos casos de borda (probabilidade próxima ao limiar) encaminhados ao time.

- **Drift de população.** Expansão para nova unidade ou mudança no perfil de pacientes pode invalidar o pipeline. Mitigação: monitoramento semanal de drift (PSI/KS) nas variáveis de maior peso (`number_inpatient`, `discharge_disposition_id`, `diag_*` agrupado, número de medicações).
- **Privacidade.** Dados clínicos sensíveis. Mitigação: treino offline sobre dados pseudo-nimizados, inferência restrita ao escopo da própria conta hospitalar, retenção mínima e sem persistência de dados crus fora de `data/raw/`.
- **Qualidade e dependência de dados.** O pipeline assume a codificação ICD-9 e os identificadores operacionais do dataset original; mudanças de padrão (por exemplo, migração para ICD-10) quebram o pré-processamento. Mitigação: validação de schema na entrada e fallback para imputação quando uma categoria nova aparece (`handle_unknown="ignore"` no OneHotEncoder).

7.4 Plano de monitoramento

- **Desempenho:** F1-macro e recall(<30) mensais sobre os pacientes com follow-up completo (rótulo disponível 30+ dias após a alta), estratificados por unidade e faixa etária.
- **Drift de dados:** PSI semanal nas 15 variáveis de maior importância; PSI > 0,2 em qualquer uma dispara revisão.
- **Métricas de produto:** taxa de aceitação do alerta pela equipe clínica, taxa de comparecimento à consulta agendada motivada pelo alerta, e latência de inferência.

7.5 Retreinamento e governança

O retreinamento é disparado quando: (a) o PSI ultrapassa 0,25 em ≥ 3 das variáveis de maior peso, (b) o F1-macro mensal de produção cai mais de 0,05 em relação ao F1-macro do teste reportado, ou (c) trimestralmente, como rotina. O pipeline de retreinamento reusa o `main.ipynb` (invalidando o cache do modelo vencedor), e o novo modelo entra em produção via *canary release* de 10% por duas semanas, com comparação A/B contra a versão vigente antes da promoção total. Cada versão de modelo é versionada junto com seu `cv_results_`, hiperparâmetros e métricas de teste, garantindo rastreabilidade e *rollback*.

7.6 Limitações

O recall padrão da classe <30 é baixo (0,196), o que torna a calibração de limiar indispensável, não opcional. A fronteira clínica de 30 dias é arbitrária, limitando o teto de desempenho independentemente do modelo (ver a discussão na Seção 5). Por fim, o modelo foi treinado em dados de 1999 a 2008 de hospitais americanos; sua transferência para outra população ou época exige revalidação antes do uso.

8 Reprodutibilidade e ferramentas

8.1 Ferramentas

- Python 3.12, gerenciado por uv (`uv.lock` versionado).

- Pipelines em `scikit-learn` 1.8 (com `imbalanced-learn` 0.14 para SMOTE).
- XGBoost 3.2 (GPU/CUDA quando disponível), LightGBM 4.6, sklvq 0.1.2 (com shim PatchedGLVQ).

8.2 Reprodução

```
uv sync
uv run jupyter nbconvert --to notebook --execute main.ipynb \
    --output main.executed.ipynb
```

A primeira execução popula `reports/cv_results/` (CSVs por modelo $\times$ variante) e `reports/figures/` (curvas, matriz, ROC/PR). Execuções subsequentes reusam o cache (`get_or_run_search`).

8.3 Uso de IA generativa

Em conformidade com a Seção 14 das exigências, declaramos o uso do assistente de programação Claude (Anthropic, Opus 4.7 e 4.8) durante o desenvolvimento, para apoio em codificação, revisão e documentação. Decisões metodológicas (split estratificado, escolha de métrica, escolha de algoritmos e variantes, leitura crítica dos resultados, e revisão final deste relatório) foram conduzidas pela equipe.